SELF-HOSTED CLOUD + GAMING STATION BAZZITE EDITION

Bazzite (Fedora Atomic) · Docker · firewalld · Fail2ban · Nginx Proxy Manager · Nextcloud AIO · DuckDNS

100% Free · Maximum Security · Permanent Home Server + Gaming Station

---

RECOMMENDED MINIMUM HARDWARE

This guide works on any x86-64 machine meeting the requirements below. Mini PCs are an excellent choice — small, quiet, low-power, and typically powerful enough for both server workloads and light gaming.

Component	Minimum	Recommended
CPU	4-core x86-64	6-core x86-64 (or better) with integrated/dedicated GPU for gaming
RAM	8 GB DDR4	16 GB DDR4 or more (gaming benefits from more RAM)
OS Drive	120 GB NVMe SSD	500 GB+ NVMe SSD (fast drive for OS + Docker)
Data Drive	Any size HDD/SSD	Dedicated drive mounted at /srv/nextcloud-data
GPU	Integrated graphics (server only)	Dedicated or eGPU for gaming (AMD plug-and-play)
Network	Gigabit Ethernet	Wired connection (Wi-Fi not recommended for servers)

◆ **NOTE:** A **dedicated data drive** is required for Nextcloud user files in this guide. It must be mounted and SELinux-labelled **before** starting Nextcloud AIO. Use lsblk to identify your drives. See Phase 4, Step 1 for setup.

STACK OVERVIEW

Component	Role & Notes
Bazzite (Host OS)	Fedora Atomic-based gaming + server OS. Immutable, auto-updating, SELinux enforcing. Boots to KDE desktop; gaming mode available on demand. Nextcloud AIO requires Docker (not Podman).
Docker + Compose	Container runtime — clean, isolated, reproducible service deployments.
firewalld	Pre-installed on Bazzite. Manages IPv4 and IPv6 via nftables. Replaces UFW.
Fail2ban	Watches SSH logs via systemd journal; bans attacking IPs through firewalld rich rules.
Nginx Proxy Manager	Handles HTTPS/SSL certificates (Let's Encrypt), routes external traffic to Nextcloud via an internal Docker network.
Nextcloud AIO	Self-hosted cloud — files, calendar, contacts, and more.
DuckDNS	Free dynamic DNS — keeps yourname.duckdns.org pointed at your home IP.
Watchtower	Keeps DuckDNS and NPM containers up to date automatically. Nextcloud AIO is updated manually.

◆ **NOTE: Future storage expansion:** When additional HDDs arrive, see Appendix E for SnapRAID + MergerFS setup — the recommended home-server storage approach at larger scales.

CONTENTS

Phase 1 — OS & Network Prep
Phase 2 — Ironclad Security (SSH Keys · Port · firewalld · Fail2ban)
Phase 3 — Docker & Services Installation
Phase 4 — Deploying Services (Network · Drive Mount · DuckDNS · NPM · Nextcloud AIO)
Phase 5 — Accessing Admin Panels via SSH Tunnel
Phase 6 — Link & Initialize
Phase 7 — Testing & Backups
Appendix A — Auto-Update Containers (Watchtower)
Appendix B — Verify System Time Sync
Appendix C — Monitor Disk Usage
Appendix D — Gaming & Console Mode
Appendix E — Storage Roadmap (SnapRAID + MergerFS)

PHASE 1 — OS & NETWORK PREP

Install Bazzite

Go to bazzite.gg and click 'Download'. Use the image selector: **Hardware = Desktop/HTPC, GPU = AMD/Intel, Desktop = KDE Plasma**, and choose the standard Desktop image (not the Steam Deck/Handheld variant). This boots to KDE by default — the right choice for a machine that is a server first, gaming station second. Gaming mode is available on demand (Appendix D).

Flash the ISO to a USB drive using Balena Etcher (free, cross-platform) or Fedora Media Writer. Boot from USB and follow the Anaconda installer. Install Bazzite to your OS drive.

■ **BAZZITE DIFFERENCE:** SSH is **not enabled by default** on Bazzite. After installation, open a terminal and run the command below before anything else. This starts the SSH server on port 22 (temporarily), allowing you to connect from your personal computer.

```
sudo systemctl enable --now sshd
```

Find Your IP Address

```
ip a   # look for inet 192.168.x.x under your ethernet interface
```

Disable Auto-Suspend (CRITICAL for servers)

■ **WARNING:** Bazzite boots to KDE Plasma, which by default will **suspend the machine after ~15 minutes of inactivity**. This silently kills all Docker containers and makes the server unreachable. You must disable this before doing anything else.

Step A — Disable in KDE (GUI): Open **System Settings** → **Power Management** → **Energy Saving**. Under the **'On AC Power'** tab, set **'Suspend session'** to **'Never'**. Apply.

Step B — Mask at the OS level (terminal, authoritative): The systemd mask is the hard OS-level guarantee. It prevents suspend regardless of KDE settings, display manager state, or future desktop configuration changes:

```
sudo systemctl mask sleep.target suspend.target \
    hibernate.target hybrid-sleep.target

# Verify — all four should show 'masked':
systemctl status sleep.target suspend.target \
    hibernate.target hybrid-sleep.target | grep -E 'Loaded:|●'
```

◆ **NOTE:** Do both steps. Step A prevents the suspend option from appearing in the KDE power menu (clean UX). Step B is the authoritative fix — it works even when logged in headlessly via SSH with no desktop session active.

Router Setup

- **Static IP (DHCP reservation):** In your router's settings, assign a permanent local IP to your server by MAC address. This ensures port-forwarding rules never break when the router reboots. Each router uses different terminology; look for 'DHCP Reservation' or 'Static Lease'.
- **Port Forwarding:** Forward these external ports to your server's local IP:
  - ■ **80 TCP** — required by Let's Encrypt for certificate issuance
  - ■ **443 TCP** — your Nextcloud's public entry point
  - ■ **53222 TCP** — SSH (our custom hardened port, configured in Phase 2)
  - ■ **3478 TCP/UDP** — required only if you plan to enable Nextcloud Talk (video/chat) later

DuckDNS

Go to duckdns.org, sign in with a Google or GitHub account, create a free subdomain (yourname.duckdns.org), and copy your Token. You will need both the subdomain name and the token in Phase 4.

PHASE 2 — IRONCLAD SECURITY

Step 1 runs on your Personal Computer. Steps 2–3 run on the Server.

Step 1 · SSH Keys (Personal Computer)

If you already have an SSH key pair, skip ssh-keygen and go straight to ssh-copy-id.

◆ **NOTE:** The -t ed25519 flag generates a modern elliptic-curve key — shorter, faster to verify, and more resistant to brute-force than the legacy RSA default.

```
ssh-keygen -t ed25519
ssh-copy-id username@192.168.1.50
# Now log in:
ssh username@192.168.1.50
```

Step 2 · SSH Config & Hardening (Server)

■ **BAZZITE DIFFERENCE:** Bazzite (Fedora) uses **socket-based SSH activation** — switching to the persistent daemon before changing the port is required. Note: Fedora names its service sshd (not ssh as Ubuntu does).

```
sudo systemctl disable --now sshd.socket
sudo systemctl enable --now sshd
```

◆ **NOTE: Why a drop-in file instead of editing sshd_config directly?** On Fedora Atomic, rpm-ostree performs a 3-way merge of /etc files during OS updates. If a future Fedora release ships a changed sshd_config and you have also edited it, a merge conflict could **silently revert your hardening settings**. Drop-in files in /etc/ssh/sshd_config.d/ are not subject to this — they are your files entirely. The main sshd_config already contains Include /etc/ssh/sshd_config.d/*.conf at its very top. Because OpenSSH uses first-match-wins for all keywords, values in the drop-in take precedence over anything in the main config.

✦ **TIP:** Before editing, confirm your exact username. Run whoami and note the output — that is the value for AllowUsers.

Create the hardening drop-in file:

```
sudo nano /etc/ssh/sshd_config.d/99-hardening.conf
```

Paste the entire block below as-is (this is a new empty file — nothing to search for):

```
Port 53222
PubkeyAuthentication yes
PasswordAuthentication no
PermitRootLogin no
MaxAuthTries 3
LoginGraceTime 20
AllowUsers YOUR_USERNAME
```

■ **WARNING:** Replace YOUR_USERNAME with your actual Bazzite login name before saving — e.g., AllowUsers john. Run whoami first if unsure. AllowUsers ensures no other account on the system can log in via SSH, even with a valid key.

Option	Value / Purpose
Port 53222	Moves SSH off port 22, eliminating the vast majority of automated scan traffic.
PubkeyAuthentication yes	Explicitly enables key login — prevents silent override by any other config.
PasswordAuthentication no	Only your SSH key can log in; passwords are fully disabled.
PermitRootLogin no	Blocks direct root access — one of the most common attack vectors.
MaxAuthTries 3 / LoginGraceTime 20	Limits the brute-force window before a connection is dropped.
AllowUsers your_name	Only your account can log in remotely. All other system users are blocked, regardless of key possession.

```
sudo sshd -t
# No output = no errors. Fix any reported problems before proceeding.
# Do NOT restart sshd yet — open the firewall port first (Step 3a below).
```

■ **WARNING:** DO NOT restart sshd until Step 3a is complete. firewalld is already active on Bazzite. If your terminal drops after sshd is restarted on port 53222 but before the firewall allows it, you will be locked out. The safe order is: **open firewall** → **restart sshd** → **test** → **remove port 22**. At every point, if your terminal drops, you can safely reconnect: before the sshd restart on port 22; after it on port 53222.

Step 3 · Firewall (Server)

■ **BAZZITE DIFFERENCE:** firewalld is pre-installed and already **active** on Bazzite. It organises rules into 'zones' applied to network interfaces. Commands marked --permanent persist across reboots but only take effect after --reload. Docker bypasses firewalld for published ports (it injects nftables rules directly), so only the SSH port requires a firewalld rule in this guide.

Step 3a — Open the new SSH port, then restart sshd:

```
# Check your active zone (usually FedoraWorkstation or public)
sudo firewall-cmd --get-default-zone

# Allow our custom SSH port FIRST
sudo firewall-cmd --permanent --add-port=53222/tcp

# Apply the firewall change
sudo firewall-cmd --reload

# NOW restart sshd — port 53222 is already open in the firewall
sudo systemctl restart sshd
```

◆ **NOTE:** Ports 80 and 443 are **NOT** added to firewalld here. Docker publishes those ports by inserting nftables rules directly, bypassing firewalld entirely. Adding them to firewalld would be redundant and misleading.

Step 3b — TEST NOW. Open a new terminal on your personal computer:

```
ssh -p 53222 username@192.168.1.50
# Do not proceed until this succeeds.
```

Step 3c — Remove port 22 only after confirming port 53222 works:

```
sudo firewall-cmd --permanent --remove-service=ssh
sudo firewall-cmd --reload

# Verify: port 22 / ssh service should be gone; 53222/tcp should be listed
sudo firewall-cmd --list-all
```

◆ **NOTE:** firewalld on Bazzite automatically enforces rules on both IPv4 and IPv6 for all configured zones. Run sudo firewall-cmd --list-all and confirm ports appear without an (ipv4) qualifier — that means they apply to all address families.

PHASE 3 — DOCKER & SERVICES INSTALLATION

Step 1 · System Auto-Updates (No action needed)

Bazzite uses an immutable OS image: the base system is read-only and updates arrive as complete new images, staged in the background. On the next reboot, the new image is activated atomically — either it works completely or the system automatically stays on the previous image. No unattended-upgrades package is needed.

If an OS update ever breaks something, one command instantly returns to the previous working state:

```
rpm-ostree rollback && systemctl reboot
```

Step 2 · Add Docker Repository

On Bazzite (Fedora Atomic), system-level software is installed with rpm-ostree, which reads package repos from /etc/yum.repos.d/. Add Docker's official Fedora repository by placing the .repo file directly — this is the most reliable method on rpm-ostree systems and matches the pattern used throughout this guide:

```
sudo curl -fsSL https://download.docker.com/linux/fedora/docker-ce.repo \
    -o  /etc/yum.repos.d/docker-ce.repo
```

◆ **NOTE:** The alternative sudo dnf5 config-manager addrepo --from-repofile=... had a real parsing bug on Fedora 40 / dnf5 ≤ 5.1.17 (issue rpm-software-management/dnf5#1603): Docker's .repo file contains comment lines that caused a fatal parse error. The bug was fixed in Fedora 41 (dnf5#1671), so current Bazzite is not affected. We use curl here because it is simpler, requires no dnf5 plugin, and is consistent with the curl pattern used in Appendix E for the mergerfs Copr repository.

◆ **NOTE:** Layering Docker via rpm-ostree is intentional here. Bazzite offers a ujust install-docker shortcut, but Nextcloud AIO strictly requires the native Docker socket (/var/run/docker.sock) for its multi-container orchestration. Rootless Podman and Distrobox workarounds lack the compatible Docker API that AIO's mastercontainer relies on. When you encounter Bazzite's official docs recommending Podman-first, understand that Nextcloud AIO is the specific exception.

Step 3 · Install Docker + Fail2ban + SELinux Tools (One Reboot)

On Fedora Atomic, rpm-ostree install creates a new OS deployment containing the layered packages. **A reboot is required** to activate it. Install everything in one command to avoid multiple reboots:

```
rpm-ostree install docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin \
    fail2ban  policycoreutils-python-utils

systemctl reboot
```

◆ **NOTE:** policycoreutils-python-utils provides the semanage command, used in Phase 4 to create permanent SELinux file-context rules for the Nextcloud data directory. This is more robust than the temporary chcon approach.

After rebooting, reconnect and enable Docker:

```
ssh -p 53222 username@192.168.1.50

# Enable Docker daemon at boot and start it now
# (Docker does NOT start automatically after install on Fedora/RPM-based systems)
sudo systemctl enable --now docker

# Allow your user to run Docker without sudo
sudo usermod -aG docker $USER

# Log out and back in for the group change to take effect
exit   # then: ssh -p 53222 username@192.168.1.50
```

Step 4 · Configure Fail2ban

Fail2ban watches the systemd journal for failed SSH authentication attempts. When an IP exceeds the threshold, it is banned via firewalld's rich rules API:

```
sudo nano /etc/fail2ban/jail.local
```

Paste exactly:

```
[sshd]
enabled   = true
port      = 53222
backend   = systemd
banaction = firewallcmd-rich-rules
```

Option	Why this value
backend = systemd	Fedora uses the systemd journal for logging (not /var/log/auth.log). This backend reads SSH failures directly from the journal — no rsyslog dependency.
banaction = firewallcmd-rich-rules	Bans are issued through firewalld's rich rules API. firewalld manages both IPv4 and IPv6, so attackers on either protocol are blocked through one consistent mechanism. This is the Bazzite/Fedora equivalent of banaction = ufw.

```
sudo systemctl enable --now fail2ban

# Jail should show as active; verify port list shows 53222, not 22
sudo fail2ban-client status sshd
```

◆ **NOTE:** Although SSH only accepts cryptographic keys (making brute-force theoretically impossible on key material), Fail2ban still prevents log flooding from bots and protects against future misconfigurations. The overhead is negligible.

Step 5 · Prevent Docker Logs from Filling Your Disk

Docker logs grow indefinitely by default and will silently fill your disk over weeks or months. Configure log rotation before starting any containers:

```
sudo nano /etc/docker/daemon.json
```

Paste:

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

```
sudo systemctl restart docker
# Each container is now capped at ~30 MB of logs total (3 files x 10 MB).
```

PHASE 4 — DEPLOYING SERVICES (DOCKER COMPOSE)

◆ **NOTE: Docker & firewalld:** Docker bypasses firewalld for published ports. When you map a port in a Compose file (e.g., -'80:80'), Docker inserts nftables rules directly — firewalld does not filter that traffic. Only map ports you explicitly intend to expose. **SELinux & Docker volumes:** Bazzite uses SELinux. Bind mounts (host directories mounted into a container) need a :z suffix so SELinux applies the correct container label. Named volumes do not need this. Nextcloud AIO's mastercontainer needs security_opt: label:disable because it must connect to the Docker socket to spawn child containers.

Step 0 · Create the Shared Docker Network

Nginx Proxy Manager and Nextcloud AIO communicate over a dedicated internal Docker network named **nextcloud-aio**. Creating it first ensures both Compose stacks can reference it as an external network. This architecture keeps the Nextcloud backend completely off the host network stack, so no ports are published to the LAN or internet for Nextcloud itself.

```
docker network create nextcloud-aio
```

◆ **NOTE:** This command only needs to be run once. Docker persists the network until you explicitly remove it. All subsequent docker compose up commands will attach containers to this existing network.

Step 1 · Mount the Data Drive (REQUIRED before Nextcloud)

■ **WARNING:** Do this before running any Nextcloud containers. The NEXTCLOUD_DATADIR path must exist, be mounted, and have the correct SELinux label before AIO's first start. It **cannot be changed** afterward without a full data migration.

```
# 1. Identify your drives
lsblk

# 2. Create a GPT partition table on the data drive
# (adjust device name from lsblk output — WARNING: destroys all existing data)
sudo parted /dev/nvme1n1 --script mklabel gpt

# 3. Create a single partition spanning the entire drive
sudo parted /dev/nvme1n1 --script mkpart primary ext4 0% 100%

# 4. Refresh the kernel's partition table view
sudo partprobe /dev/nvme1n1

# 5. Format the partition (note: nvme1n1p1, not nvme1n1)
sudo mkfs.ext4 /dev/nvme1n1p1

# 6. Create the mount point
sudo mkdir -p /srv/nextcloud-data

# 7. Apply a permanent SELinux file-context rule
sudo semanage fcontext -a -t container_file_t "/srv/nextcloud-data(/.*)?"
sudo restorecon -Rv /srv/nextcloud-data

# 8. Get the partition UUID for stable fstab mounting
sudo blkid /dev/nvme1n1p1

# 9. Add to /etc/fstab for automatic mounting at boot
sudo nano /etc/fstab
# Add this line (replace UUID with the output from blkid):
# UUID=your-uuid  /srv/nextcloud-data  ext4  defaults  0  2

# 10. Mount now without rebooting
sudo mount -a

# 11. Verify
df -h /srv/nextcloud-data
```

◆ **NOTE:** Why semanage fcontext instead of chcon? chcon applies a label to existing files but the label is lost if the filesystem is relabelled (which can happen during Fedora Atomic OS updates). semanage fcontext writes a persistent policy rule; restorecon then applies it. Any newly created files under /srv/nextcloud-data automatically inherit the container_file_t type, and future relabels will correctly restore it.

Step 2 · DuckDNS Auto-Updater

Keeps your domain pointed at your home IP. No bind mounts — no SELinux changes needed.

```
mkdir -p ~/docker/duckdns && cd ~/docker/duckdns
nano docker-compose.yml
```

docker-compose.yml:

```
services:
  duckdns:
    image: lscr.io/linuxserver/duckdns:latest
    container_name: duckdns
    restart: unless-stopped
    labels:
      -  "com.centurylinklabs.watchtower.enable=true"
    environment:
      - SUBDOMAINS=yourname    # subdomain only, not the full domain
      - TOKEN=your-token-here
      - TZ=Europe/Berlin       # e.g. America/New_York, Asia/Tokyo

docker compose up -d
```

Step 3 · Nginx Proxy Manager (NPM)

Handles SSL certificates and routes external traffic to Nextcloud. The :z suffix on bind mount volumes tells SELinux to apply the container label. Port 81 (admin panel) is bound to 127.0.0.1 — accessible only via SSH tunnel (Phase 5).

NPM is attached only to the nextcloud-aio network. User-defined Docker bridge networks have outbound internet access via host NAT masquerading by default, so no second 'default' network is needed for Let's Encrypt certificate negotiation.

```
mkdir -p ~/docker/npm && cd ~/docker/npm
nano docker-compose.yml
```

docker-compose.yml:

```
services:
  app:
    image: 'jc21/nginx-proxy-manager:latest'
    container_name: nginx-proxy-manager
    restart: unless-stopped
    labels:
      - "com.centurylinklabs.watchtower.enable=true"
    ports:
      - '80:80'
      - '443:443'
      - '127.0.0.1:81:81'    # admin panel: localhost only
    networks:
      - nextcloud-aio         # internet access + internal path to Nextcloud
    volumes:
      - ./data:/data:z            # :z = SELinux shared label
      - ./letsencrypt:/etc/letsencrypt:z

networks:
  nextcloud-aio:
    external: true             # uses the network created in Step 0

docker compose up -d
```

Step 4 · Nextcloud All-in-One

Nextcloud AIO requires Docker. It orchestrates multiple child containers by connecting to the Docker socket. This is why Podman is not a supported alternative — its Docker API compatibility is not sufficient for AIO's multi-container management.

The mastercontainer is placed on the nextcloud-aio network. AIO will also put all its child containers (including nextcloud-aio-apache) on this network, making them reachable from NPM by container name. With APACHE_IP_BINDING=127.0.0.1, port 11000 is published only on the host's localhost interface, keeping it off the LAN and the internet.

■ **WARNING:** Set NEXTCLOUD_DATADIR **before** the first start. This environment variable tells AIO where to store all user files. It **CANNOT** be changed after initial setup without a full data migration.

```
mkdir -p ~/docker/nextcloud && cd ~/docker/nextcloud
nano docker-compose.yml
```

docker-compose.yml:

```
volumes:
  nextcloud_aio_mastercontainer:
    name: nextcloud_aio_mastercontainer

services:
  nextcloud-aio-mastercontainer:
    image: ghcr.io/nextcloud-releases/all-in-one:latest
    init: true
    restart: always
    container_name: nextcloud-aio-mastercontainer
    security_opt:
      - label:disable   # SELinux: allows Docker socket access for this container only
    labels:
      - "com.centurylinklabs.watchtower.enable=false"
    volumes:
      - nextcloud_aio_mastercontainer:/mnt/docker-aio-config
      - /var/run/docker.sock:/var/run/docker.sock:ro
    ports:
      - '127.0.0.1:8080:8080'   # setup panel: localhost only
    networks:
      - nextcloud-aio            # shared with NPM for internal routing
    environment:
      - APACHE_PORT=11000
      - APACHE_IP_BINDING=127.0.0.1   # port 11000 on localhost only
      - NEXTCLOUD_DATADIR=/srv/nextcloud-data
      # Uncomment the line below ONLY if your router lacks NAT loopback
      # and neither router DNS override nor Pi-hole is an option.
      # See Phase 6, Step 2 for the full explanation.
      # - SKIP_DOMAIN_VALIDATION=true

networks:
  nextcloud-aio:
    external: true

docker compose up -d
```

◆ **NOTE:** Why APACHE_IP_BINDING=127.0.0.1 instead of 0.0.0.0? With the shared Docker network, NPM reaches the nextcloud-aio-apache container directly by name (nextcloud-aio-apache:11000) without going through the host network stack. Setting APACHE_IP_BINDING=127.0.0.1 means AIO publishes port 11000 only on the host's loopback interface, preventing LAN or IPv6 access. This removes the need to disable IPv6 system-wide.

◆ **NOTE:** The official image is hosted on GHCR: ghcr.io/nextcloud-releases/all-in-one:latest. This is the canonical registry per the official Nextcloud AIO repository. security_opt: label:disable is required on SELinux-enforcing systems (Bazzite/Fedora) to allow the mastercontainer to access the Docker socket and spawn child containers.

◆ **NOTE: Why :ro on the Docker socket?** Mounting a Unix socket as :ro does *not* mean the container can only read data. It means the container cannot delete or replace the socket file itself on the host filesystem. All bidirectional API communication — including starting and stopping child containers — still flows through it normally. The :ro flag is a host-filesystem protection, not a communication restriction. Do not remove it.

PHASE 5 — ACCESSING ADMIN PANELS (SSH TUNNEL)

The admin panels (ports 81 and 8080) are bound to 127.0.0.1 — invisible to your local network. Access them by creating an SSH tunnel from your personal computer. Run the relevant command and leave it running while you use the panel.

Run on your Personal Computer:

```
# NPM admin panel  →  http://localhost:8081
ssh -N -L 8081:127.0.0.1:81 -p 53222 username@192.168.1.50

# Nextcloud AIO setup panel  →  https://localhost:9443
ssh -N -L 9443:127.0.0.1:8080 -p 53222 username@192.168.1.50
```

◆ **NOTE:** -N means 'do not execute a remote command' (tunnel only). -L binds a local port to a remote address through the encrypted SSH connection. Port 53222 is our hardened SSH port. Re-run the relevant tunnel command each time you need to access an admin panel.

PHASE 6 — LINK & INITIALIZE

Step 1 · Configure Nginx Proxy Manager

Start the NPM tunnel (first command in Phase 5), then open http://localhost:8081 in your browser. Log in with **admin@example.com / changeme** — change both immediately after logging in.

Go to **Hosts** → **Proxy Hosts** → **Add Proxy Host** and fill in:

Field	Value
Domain Names	yourname.duckdns.org
Scheme	http
Forward Hostname / IP	nextcloud-aio-apache
Forward Port	11000
Toggles	Cache Assets, Block Common Exploits, Websockets Support — all ON
SSL tab	Request new SSL Certificate · Force SSL · HTTP/2 Support · Agree to ToS → Save

◆ **NOTE:** The Forward Hostname is the Docker container name nextcloud-aio-apache, not host.docker.internal. NPM and the Nextcloud backend are on the same nextcloud-aio Docker network, so NPM resolves the container name directly. NPM saves the proxy host configuration regardless of whether the backend is running — a missing backend only causes a 502 error when the domain is visited, not during setup. If you see an error while saving, it indicates a Let's Encrypt certificate issuance failure — typically caused by ports 80 or 443 not being forwarded from your router.

Step 2 · Fix Local Network Access (NAT Loopback)

■ **WARNING:** Complete this step **before** entering your domain in Step 3. Nextcloud AIO performs its domain validation by spawning a temporary nextcloud-aio-domaincheck container that makes an HTTP request from inside Docker to port 443 of your domain. If your router does not support NAT loopback (hairpin NAT), this container-side request will fail and you will not be able to proceed, regardless of what you configure on client devices.

Many routers cannot route traffic that originates on the local network back through the router's own public IP. The result: yourname.duckdns.org resolves to your public IP, but the connection cannot loop back — it fails from inside your home network even though it works fine from the internet.

Solution 1 — Router local DNS override (recommended): In your router's settings, look for 'Local DNS', 'DNS Entries', or 'Static Hosts' and add:

```
yourname.duckdns.org  →  192.168.1.50
```

This makes all devices on your LAN resolve the domain directly to the server's local IP, bypassing the public IP entirely. External devices continue using DuckDNS normally. No per-device configuration needed.

Solution 2 — Pi-hole local DNS: If your router does not support custom DNS entries, run Pi-hole as a local DNS resolver and add the same override there. Pi-hole's 'Local DNS Records' feature handles this cleanly.

■ **WARNING: Client hosts files do NOT solve this.** Adding yourname.duckdns.org to /etc/hosts on your PC only helps your browser. It has no effect on the domain validation request, which originates from *inside the Docker container on the server* — not from your client device. This is a common and permanent point of confusion — do not attempt it.

Solution 3 — SKIP_DOMAIN_VALIDATION (last resort): If neither router DNS nor Pi-hole is available, you can bypass the container-side check entirely. Edit the Nextcloud AIO compose file, uncomment the SKIP_DOMAIN_VALIDATION line added in Phase 4 Step 4, then recreate the container:

```
cd ~/docker/nextcloud

# Edit the compose file and uncomment:
# - SKIP_DOMAIN_VALIDATION=true
nano docker-compose.yml

# Recreate the container to apply the new environment variable:
docker compose down && docker compose up -d
```

◆ **NOTE:** SKIP_DOMAIN_VALIDATION=true bypasses only the AIO setup-panel check. Your Nextcloud instance still needs to be genuinely reachable at port 443 for actual operation and SSL certificate issuance — this variable does not fix a broken network configuration; it only removes the pre-flight gate. Use Solution 1 or 2 if at all possible.

Step 3 · Initialize Nextcloud

Stop the NPM tunnel (Ctrl+C). Start the Nextcloud AIO tunnel (second command in Phase 5). Go to https://localhost:9443 and accept the browser security warning — this is expected.

◆ **NOTE:** The warning is about AIO's self-signed certificate, not your real domain cert. The SSH tunnel is an encrypted pipe; the self-signed cert comes from the AIO container. Safe to proceed.

- Copy the setup password shown on screen.
- Enter your domain: yourname.duckdns.org
- Data directory is already set via NEXTCLOUD_DATADIR — no change needed.
- Choose optional containers and click **Start Containers**.
- Wait 5–10 minutes for Nextcloud to pull and configure all components. AIO will create the nextcloud-aio-apache container and attach it to the nextcloud-aio network, making it reachable from NPM.

PHASE 7 — TESTING & BACKUPS

DNS Test

```
nslookup yourname.duckdns.org 8.8.8.8
```

The 8.8.8.8 argument queries Google's public DNS directly, bypassing the local DNS override set in Phase 6 Step 2. Confirm the result is your home's public IP.

External Test

Turn off Wi-Fi on your phone and visit https://yourname.duckdns.org over cellular. A green padlock and the Nextcloud login screen means everything is working.

Backups

Inside the AIO setup panel (https://localhost:9443) you will find the built-in **Borg Backup** tool. Do not skip this.

- Set a destination path (e.g., /mnt/backups or a mounted external USB drive).
- Set backups to run daily.

■ **WARNING:** Perform a **test restore once a month**. A backup you have never tested cannot be trusted.

APPENDIX A — AUTO-UPDATE CONTAINERS (WATCHTOWER)

Docker containers do not update themselves — outdated containers accumulate vulnerabilities. Watchtower automates this using opt-in mode (--label-enable): it only touches containers explicitly marked with enable=true — DuckDNS and NPM. All Nextcloud AIO containers are intentionally excluded.

```
mkdir -p ~/docker/watchtower && cd ~/docker/watchtower
nano docker-compose.yml
```

docker-compose.yml:

```
services:
  watchtower:
    image: containrrr/watchtower
    restart: unless-stopped
    security_opt:
      - label:disable   # SELinux: allows Docker socket access
    environment:
      - TZ=Europe/Berlin   # match your DuckDNS container timezone
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    command: --label-enable --cleanup --schedule "0 0 4 * * *"
    # Only updates containers with enable=true (DuckDNS, NPM).
    # All Nextcloud AIO containers are untouched. Runs at 04:00 local time.

docker compose up -d
```

Updating Nextcloud AIO manually: Open the AIO setup panel (https://localhost:9443) and use the built-in update button when a new version is available.

APPENDIX B — VERIFY SYSTEM TIME SYNC

Incorrect system time silently breaks TLS certificate validation and Nextcloud login sessions. Bazzite (Fedora Atomic) uses **chronyd** as its default NTP daemon — not systemd-timesyncd, which is a Ubuntu/Debian default and is not enabled by default on Fedora.

```
timedatectl
# Confirm: System clock synchronized: yes

# Check the chrony service status:
systemctl status chronyd

# If chronyd is not running, enable it:
sudo systemctl enable --now chronyd
```

■ **WARNING:** Do **not** attempt to enable systemd-timesyncd on Bazzite. It is not the active NTP implementation on Fedora-based systems, and enabling it while chronyd is present can cause a conflict resulting in time drift. Always use chronyd on Bazzite/Fedora.

APPENDIX C — MONITOR DISK USAGE

A full disk will crash your database without warning. Check periodically:

```
df -h

# When any partition hits 90%+ usage, act immediately:
# Clean old Docker images:   docker system prune
# Move data to external/HDD storage, or expand capacity.
```

APPENDIX D — GAMING & CONSOLE MODE (BAZZITE-SPECIFIC)

Bazzite ships with all gaming infrastructure already in place. Server containers run entirely independently of the desktop session — opening a game does not pause, slow, or otherwise affect your running Docker services. The container engine and all restart: always containers are managed by systemd at the OS level, not tied to any user session.

Steam

■ **BAZZITE DIFFERENCE:** Steam (and Lutris) are pre-installed as part of the Bazzite Desktop image — they are baked into the OS, not added via Flatpak. Do **not** run flatpak install flathub com.valvesoftware.Steam; this creates a redundant duplicate installation with conflicting desktop shortcuts and Flatpak sandbox limitations. Simply open the pre-installed Steam application from your KDE application menu.

TV / Console Experience (Steam Big Picture Mode)

Connect your server to the TV via HDMI. Launch Steam, then go to **View** → **Big Picture Mode** for a full couch gaming interface. To start Steam in Big Picture automatically: Steam → Settings → Interface → 'Start Steam in Big Picture Mode on launch'. When you are done gaming, exit Big Picture — the KDE desktop is immediately available for server management. All containers continue running throughout.

eGPU Support

AMD Radeon RX 6000/7000 series eGPUs are plug-and-play on Bazzite with zero driver configuration. Bazzite auto-detects and uses the eGPU for games and AI inference (Ollama + ROCm). To explicitly route a specific game to the eGPU, add this Steam launch option:

```
DRI_PRIME=1 %command%
```

OS Updates and Reboots

Bazzite stages OS updates silently in the background. Applying them requires a reboot — during which your server is offline for 1–2 minutes. All containers come back up automatically on reboot (restart: always / restart: unless-stopped). Schedule reboots during low-usage hours. If an update causes any problem:

```
rpm-ostree rollback && systemctl reboot
```

APPENDIX E — STORAGE ROADMAP (SNAPRAID + MERGERFS)

This section describes a recommended storage expansion path when you add additional HDDs. It does not apply to initial setup.

Drive	Current Role
OS Drive (fast NVMe, recommended Gen 4)	Bazzite OS + Docker engine + container images + databases.
Data Drive (secondary SSD or HDD)	Nextcloud user files at /srv/nextcloud-data. Also holds Borg backups until additional HDDs arrive.

Why SnapRAID + MergerFS (not RAID1 or ZFS)

For a home server with mixed-size drives, SnapRAID + MergerFS is the recommended approach:

- **Works with mixed-size drives:** unlike RAID1 or traditional RAID5/6, you can combine drives of different sizes in the same pool freely.
- **No continuous write overhead:** parity is computed once daily via a scheduled sync, not on every write. Ideal for HDDs and preserves drive longevity.
- **Files remain independently accessible:** each file lives as-is on exactly one drive. If a non-parity drive fails, only files on that drive are at risk — every other drive is immediately readable without reconstruction.
- **Easy expansion:** add a new drive, run a sync, and it is part of the pool. No need to reformat or rebuild the array.
- **Transparent recovery:** because files are stored as-is (not striped), recovery tools work on individual drives even if MergerFS or SnapRAID is unavailable.

- **Lower complexity than ZFS:** ZFS is an excellent choice (especially with ample RAM for ARC cache), but SnapRAID + MergerFS has fewer moving parts and is simpler to maintain at home-server scale.

> ◆ **NOTE:** ZFS is a strong alternative with real-time checksums, snapshots, and native RAID-Z. The commonly cited '1 GB of RAM per 1 TB of storage' rule is a guideline for ARC cache sizing, not a hard requirement. With sufficient RAM, ZFS becomes an excellent choice. SnapRAID + MergerFS remains the simpler option for most home setups.

Migration Plan (once additional HDDs arrive)

```
# 1. Add the mergerfs Copr repository
#    mergerfs is NOT in the official Fedora repos — install from the community Copr.
#    The subshell $(rpm -E %fedora) auto-inserts your Fedora version number.
sudo curl -fsSL \
    "https://copr.fedorainfracloud.org/coprs/errornointernet/mergerfs/repo/fedora-$(rpm  -E  %fedora)/errornointern
    -o /etc/yum.repos.d/mergerfs.repo
sudo rpm-ostree refresh-md --force

# 2. Install MergerFS + SnapRAID (snapraid is in the official EPEL/Fedora repos)
rpm-ostree install mergerfs snapraid
systemctl reboot

# 3. Format and configure your HDDs (parted + mkfs.ext4 as in Phase 4 Step 1)

# 4. Set up MergerFS unified pool: /mnt/disk1 + /mnt/disk2 -> /srv/media

# 5. Configure SnapRAID with one disk as parity

# 6. Stop Nextcloud AIO containers
cd ~/docker/nextcloud && docker compose stop

# 7. Migrate data
sudo rsync -aHAX /srv/nextcloud-data/ /srv/media/nextcloud/

# 8. Update NEXTCLOUD_DATADIR in compose file to new path
nano ~/docker/nextcloud/docker-compose.yml
# Change: NEXTCLOUD_DATADIR=/srv/media/nextcloud

# 9. Apply SELinux label to new path
sudo semanage fcontext -a -t container_file_t "/srv/media/nextcloud(/.*)?"
sudo restorecon -Rv /srv/media/nextcloud

# 10. Reinitialise AIO against the new path
docker compose up -d

# 11. Repurpose original data drive for game library or Ollama AI model storage
```